AREHI-SECOPS

ASEAN Regional E-Commerce Hybrid Infrastructure & Hardened SecOps System

Phase 3 — Security Hardening & IP Services

Full-Depth Configuration & Certification Study Guide

Architect / Author: Danial Nur Irfan bin Azeze
Bachelor of Computer Science — Networks & Security, UTM
CompTIA Security+ Certified

Exam objectives referenced: Cisco CCNA 200-301, CompTIA Security+ SY0-701, AWS Certified Solutions Architect – Associate (SAA-C03)
Lab platform: Cisco Packet Tracer 9.0.0 (device config) + real OpenSSL (PKI)

Devices in scope: all 6 —
MY-KL-HQ-CORE (Catalyst 3650-24PS), SG-EDGE-GW (ISR 4331),
PH-MNL-ROAS / TH-BKK-ROAS (ISR 4321),
PH-MNL-ACC / TH-BKK-ACC (Catalyst 2960-24TT)

Contents

1. Phase 3 Overview

Phase 3 shifts focus from "does the network work" (Phases 1–2) to "is the network defensible." It covers IP services that make the network operable at scale (NTP, SNMP, DHCP relay, NAT, config backups), Layer 2 active defense against two specific attack classes (rogue DHCP servers, ARP poisoning), Layer 3 traffic control (ACLs), and a real Public Key Infrastructure build — the most direct CompTIA Security+ content in the whole project so far.

Devices in scope

Hostname	Confirmed PT model	Phase 3 objectives touching it
MY-KL-HQ-CORE	Catalyst 3650-24PS	NTP, SNMP, DHCP Snooping/DAI, DHCP Relay, ACLs (standard + extended)
SG-EDGE-GW	ISR 4331	NTP, SNMP, NAT, ACL (standard only — no local GUEST_WIFI to contain)
PH-MNL-ROAS	ISR 4321	NTP, SNMP, DHCP Relay, ACLs (standard + extended)
TH-BKK-ROAS	ISR 4321	NTP, SNMP, DHCP Relay, ACLs (standard + extended)
PH-MNL-ACC	Catalyst 2960-24TT	NTP, SNMP, DHCP Snooping/DAI, ACL (standard only, IPv4)
TH-BKK-ACC	Catalyst 2960-24TT	NTP, SNMP, DHCP Snooping/DAI, ACL (standard only, IPv4)

2. A Note on Exam Objective Tags

Phase 3 is the most evenly split phase across all three certifications. **CCNA 200-301 Domain 4.0 (IP Services)** covers NTP, SNMP, DHCP relay, NAT, and config backups directly; **Domain 5.0 (Security Fundamentals)** covers DHCP Snooping, DAI, and ACLs directly. **Security+ SY0-701** appears heavily here too — this phase is where actual attack types (rogue DHCP, ARP poisoning) and actual PKI concepts get built, not just referenced. **AWS SAA-C03** appears twice, both genuinely: NAT (Objective 7, conceptually mirroring what an AWS NAT Gateway does for a private subnet) and PKI (Objective 9, since the certificate built here is intended to eventually terminate on the AWS ALB in Phase 4).

3. Objective 1 — Planning: Server Inventory & ACL Policy Matrix

Planning only — no device commands, produced the reference plan the rest of Phase 3 is built from

CCNA 200-301: Domain 5.0 (Security Fundamentals) Security+ SY0-701: Domain 3.0 (Security Architecture)

Purpose: decide two things up front, the same way the IPv4/IPv6 addressing plans were built before any device typing began in Phases 1–2: where do the various services (DHCP, TFTP, SNMP traps) actually point to, and what should the ACL policy actually enforce.

Placeholder server inventory

No server devices are modeled in this Packet Tracer topology, so every IP service in this phase points at one consolidated placeholder address in Malaysia HQ's `DMZ_SERVERS` VLAN — matching that VLAN's own stated purpose ("TFTP / SIEM / RADIUS").

Service	Address
Core services server (DHCP, TFTP, Syslog/SIEM, RADIUS, SNMP)	10.10.40.10
NTP source	`MY-KL-HQ-CORE` itself, as `ntp master`

ACL policy design

ACL	Type	Enforces
MGMT-SSH-ONLY	Standard, IPv4 + IPv6	Only traffic sourced from a MGMT subnet (any of the 3 sites) can even attempt SSH — applied on every device's VTY lines
GUEST-CONTAINMENT	Extended, IPv4 + IPv6	GUEST_WIFI at any site is blocked from reaching MGMT, LOGISTICS_SALES, or DMZ_SERVERS at any of the 3 sites — applied inbound where each site's guest traffic first reaches a router

EXAM ANGLE — STANDARD VS EXTENDED ACLS

A **standard ACL** can only match on source IP address — nothing else. That's exactly why it's the right tool for "only these subnets may attempt SSH," since the decision only ever needs to depend on where the traffic came from. An **extended ACL** can match source AND destination address, protocol, and port — necessary for GUEST-CONTAINMENT, since that policy needs to evaluate both where traffic is coming from and where it's going.

4. Objective 2 — NTP

All 6

CCNA 200-301: Domain 4.0 (IP Services) Security+ SY0-701: Domain 4.0 (Security Operations — log correlation integrity)

Purpose: consistent time across every device matters more than it sounds — accurate, synchronized timestamps are what make Syslog entries, SNMP traps, and SSH login records actually useful for correlating an incident afterward. Comparing logs between two devices with drifted clocks during a security investigation is unreliable at best.

Commands, word by word

```
! MY-KL-HQ-CORE (the master):
ntp master 3

! Every other device:
ntp server 10.255.255.1
```

Word / term	Meaning
`ntp master 3`	Makes this device an authoritative NTP time source for others to sync against. The `3` is the **stratum** it claims — stratum 1 is reserved for devices directly attached to a real reference clock (GPS receiver, atomic clock radio, etc.); claiming stratum 3 is an honest admission this is not a true external authority, while still being usable as one internally.
Stratum	A number describing how many hops away from a real reference clock a given NTP source is — stratum 1 is closest, and it increases by exactly 1 with each additional hop away (a client syncing to a stratum-3 master itself becomes stratum 4).
`ntp server 10.255.255.1`	Points this device at `MY-KL-HQ-CORE`'s loopback as its time source — reachable from every device now thanks to Phase 2's OSPF, even the two access switches (via their default-gateway route through their local ROAS router).

EXPECTED RESULT

```
show ntp status
show ntp associations
```

On `MY-KL-HQ-CORE`: `show ntp status` reports synchronized, stratum 3, referencing `127.127.1.1` (NTP's special internal-loopback address meaning "I am my own source"). On every client: stratum should be exactly one higher than whatever it's syncing against, confirming the hierarchy is working correctly.

PLATFORM QUIRK CONFIRMED IN THIS LAB

`show ntp associations` on `SG-EDGE-GW` showed two entries: the configured one (`10.255.255.1` , the loopback) sitting at stratum 16 (NTP's special "unsynchronized" value, never actually reachable), and an unconfigured second entry (`10.10.254.1` — `MY-KL-HQ-CORE` 's directly-connected physical WAN interface) that was fully synced and marked as the active selected source. This simulator appears to auto-discover/peer with a directly-connected `ntp master` via the physical interface, somewhat independently of the exact IP given to `ntp server` . The practical outcome — the device genuinely ends up synchronized, confirmed via `show ntp status` reporting the correct stratum — is what actually matters; the specific-peer-address discrepancy was not chased further.

5. Objective 3 — SNMPv3 (and a Platform Limitation)

All 6

CCNA 200-301: Domain 4.0 (IP Services)

Security+ SY0-701: Domain 2.0 & 4.0 (secure protocols — v3 authentication/encryption vs v1/v2c cleartext)

Purpose: SNMPv3 is the secure version of SNMP. Versions 1 and 2c send "community strings" — effectively passwords — in cleartext, a well-known, frequently tested security weakness. Version 3 adds real authentication and encryption, which is exactly why the design specifically calls for v3 rather than just "SNMP."

Intended design, word by word

```
snmp-server group NETADMINS v3 priv
snmp-server user netmonitor NETADMINS v3 auth sha AuthPass123 priv aes 128 PrivPass123
snmp-server host 10.10.40.10 version 3 priv netmonitor
snmp-server enable traps
```

Word / term	Meaning
`snmp-server group ... v3 priv`	Creates an SNMPv3 group requiring `priv` — the strongest of SNMPv3's three security levels (`noauth` , `auth` , `auth priv`), meaning both authentication and encryption are mandatory for anyone in this group.
`auth sha`	SHA as the hashing algorithm used to authenticate the SNMP user, verifying messages actually came from who they claim to.
`priv aes 128`	AES-128 as the encryption algorithm protecting the actual SNMP payload content in transit.
`snmp-server host ... version 3 priv`	Where unsolicited trap notifications (e.g. "a port just went down") get sent, and at what security level.

CONFIRMED PLATFORM LIMITATION — THIS SIMULATOR CANNOT REPRESENT SNMPV3 AT ALL

`snmp-server ?` on this platform (confirmed on `MY-KL-HQ-CORE`) returns exactly one keyword: `community` . No `group` , `user` , `host` , or `enable traps` — meaning there is no way to represent SNMPv3's group/user security model on this platform, not even as something automatically active in the background. This differs from earlier cases like `spanning-tree extend system-id` (Phase 1), where the underlying feature was genuinely present just not configurable — here, the feature simply does not exist in this simulator at all.

Resolution (explicitly decided): the reference configuration files keep the full SNMPv3 design above unchanged, since it stays accurate to real Catalyst/ISR hardware, which does fully support v3. For the live Packet Tracer lab, the achievable fallback was used on all 6 devices instead:

```
snmp-server community AREHI-RO-2026 RO
```

This is a genuine security regression from the design intent — plaintext community-string polling instead of authenticated, encrypted v3 — made explicit and deliberate rather than silently accepted, and worth stating clearly in any portfolio discussion of this project: the gap is the lab platform's limitation, not a misunderstanding of why v3 matters.

EXPECTED RESULT (LAB FALLBACK)

```
show running-config | include snmp
```

No dedicated `show snmp` command exists on this platform either — verifying directly against the running configuration is the correct fallback here, same pattern used whenever a dedicated verification command turned out not to exist.

6. Objective 4 — DHCP Snooping & Dynamic ARP Inspection

MY-KL-HQ-CORE, PH-MNL-ACC, TH-BKK-ACC

CCNA 200-301: Domain 4.0 (IP Services) & Domain 5.0 (Security Fundamentals)

Security+ SY0-701: Domain 2.0 (attack types — rogue DHCP, ARP poisoning/spoofing)

Purpose: two related but distinct Layer 2 attacks, both defended by the same underlying trust concept.

- **Rogue DHCP server attack:** an unauthorized device plugged into an access port can answer DHCP requests itself, handing out a malicious default gateway or DNS server and silently redirecting victims' traffic.
- **ARP poisoning / man-in-the-middle:** ARP has no authentication whatsoever — any device can claim to own any IP address, redirecting traffic meant for the real gateway through the attacker instead.

Commands, word by word

```
ip dhcp snooping
ip dhcp snooping vlan 10,20,30,40
ip arp inspection vlan 10,20,30,40
ip arp inspection validate src-mac dst-mac ip
!
interface GigabitEthernet0/2
 ip dhcp snooping trust
 ip arp inspection trust
```

Word / term	Meaning
DHCP Snooping	Builds a table of legitimate IP-to-MAC-to-port bindings by only trusting DHCP server responses (`DHCPOFFER` / `DHCPACK`) arriving on ports explicitly marked trusted. Any such response arriving on an untrusted port (the default state for every port) is dropped.
Dynamic ARP Inspection (DAI)	Cross-references every ARP packet against the same binding table DHCP Snooping built, dropping any ARP packet whose claimed IP-to-MAC pairing doesn't match what was legitimately observed at DHCP lease time.
`ip arp inspection validate src-mac dst-mac ip`	Adds extra rigor beyond the basic binding-table check: also validates the Ethernet header's source/destination MAC against the ARP payload's claimed MAC, and checks the IP fields for consistency — catches more spoofing variants than the base check alone.
Trusted vs untrusted port	Trusted ports (here, the uplink to the ROAS router) are where legitimate DHCP responses and real gateway ARP traffic are expected to arrive from. Every end-host-facing port stays untrusted (the default) — exactly where a rogue device would actually be plugged in.

EXPECTED RESULT

```
show ip dhcp snooping
show ip arp inspection
```

Confirms the right VLANs are active for both features, and that the uplink trunk shows `trusted` while every other port shows `untrusted` .

7. Objective 5 — DHCP Relay

MY-KL-HQ-CORE, PH-MNL-ROAS, TH-BKK-ROAS

CCNA 200-301: Domain 4.0 (IP Services)

Purpose: a DHCP client's initial request (`DHCPDISCOVER`) is a Layer 2 broadcast, which never leaves its own VLAN on its own. Since the placeholder DHCP server sits in a completely different VLAN than the requesting clients, something must catch that broadcast and forward it as a unicast packet toward the server — that's `ip helper-address`, configured on each VLAN's first-hop gateway interface.

Commands, word by word

```
interface Vlan20
 ip helper-address 10.10.40.10
```

Word / term	Meaning
`ip helper-address`	Converts a broadcast DHCP request received on this interface into a unicast packet destined for the specified server address, then relays the server's reply back the same way — this is literally what makes the term "DHCP *relay*" accurate.

Scope reference

VLAN	Gets relay?	Why
10 (MGMT)	No	Infrastructure/admin subnet — would use static addressing in a real deployment
20 (LOGISTICS_SALES)	Yes	Real end-user staff workstations request leases here
30 (GUEST_WIFI)	Yes	Real guest devices request leases here
40 (DMZ_SERVERS)	No	Infrastructure subnet — servers use static addressing

EXPECTED RESULT

```
show run interface Vlan20
```

Since no real DHCP server device exists in this lab to actually respond, this configuration-level check — confirming the `ip helper-address` line is present and correct — is the appropriate verification here,

> the same situation as SNMP and RSA keys earlier in the project.

8. Objective 6 — TFTP Configuration Backup

All 6 (demonstrated as an EXEC workflow, not stored configuration)

CCNA 200-301: Domain 4.0 (IP Services — IOS configuration/image management)

Purpose: back up a device's running configuration to an external server, so a configuration can be restored after a device failure or a bad change, without needing physical console access to retype everything from scratch.

Command, word by word

```
copy running-config tftp:
Address or name of remote host []? 10.10.40.10
Destination filename [my-kl-hq-core-confg]? my-kl-hq-core-2026-07-26.cfg
```

Word / term	Meaning
`copy running-config tftp:`	An **interactive EXEC command**, not a configuration line — it never appears in `show running-config`, the same way `crypto key generate rsa` never does. It prompts for the remote host and filename rather than taking them as arguments.
TFTP (Trivial File Transfer Protocol)	A simple file-transfer protocol running over UDP, historically used for exactly this kind of network-device configuration/image transfer — "trivial" because it has no authentication mechanism at all, which is why it should only ever be used inside a trusted internal segment (here, the DMZ_SERVERS VLAN), never across an untrusted network.

EXPECTED RESULT

IOS confirms the transfer with an `OK` message and a byte count once it completes successfully. Including a date in the destination filename makes it obvious which backup is most recent if this is ever repeated later in the project's life.

9. Objective 7 — NAT (PAT Overload + Static NAT)

SG-EDGE-GW only

CCNA 200-301: Domain 4.0 (IP Services) AWS SAA-C03: Design Cost-Effective/Resilient Architectures (NAT Gateway concept)

Purpose: this router sits at the boundary between the internal `10.10.0.0/16` private address space and the public-facing placeholder interface toward the internet/AWS. NAT translates between the two, since private (RFC 1918-style) addressing is never valid on the real internet.

Commands, word by word

```
interface GigabitEthernet0/0/0
 ip nat inside
!
interface GigabitEthernet0/0/1
 ip nat outside
!
ip access-list standard NAT-INSIDE-HOSTS
 permit 10.10.0.0 0.0.255.255
!
ip nat inside source list NAT-INSIDE-HOSTS interface GigabitEthernet0/0/1 overload
ip nat inside source static 10.10.40.10 203.0.113.3
```

Word / term	Meaning
`ip nat inside` / `ip nat outside`	Every NAT command depends on these designations existing first — IOS needs to know which interface represents the private network side and which represents the public side before any translation logic can be applied.
Dynamic PAT / overload	**Port Address Translation** — many internal hosts share the router's single public IP simultaneously, each one distinguished by a unique combination of source port. `overload` is the specific keyword meaning many-to-one, as opposed to a one-to-one dynamic pool.
Static NAT	A fixed, permanent one-to-one address mapping — used here for the DMZ server specifically because inbound services (a web app, for instance) need a consistent, predictable public address rather than whatever port PAT happens to assign moment to moment.

EXAM ANGLE — A COMMON MISCONCEPTION WORTH STATING DIRECTLY

NAT is not a firewall and provides no real access control on its own — it only translates addresses. Any actual security benefit (hiding internal addressing structure from the outside) is a side effect, not NAT's purpose. This distinction between "NAT hides addresses" and "NAT provides security" is a frequently tested point across both CCNA and Security+.

EXPECTED RESULT

```
show ip nat translations
show ip nat statistics
```

With no real internet-side device generating inbound traffic yet (that's Phase 4's AWS work), dynamic PAT entries won't appear until real traffic triggers them — but the **static** entry should show up immediately, since it doesn't require any traffic to activate. That static entry is the actual proof this objective worked.

10. Objective 8 — Standard & Extended ACLs (IPv4 + IPv6)

All 6 (standard ACL); MY-KL-HQ-CORE, PH-MNL-ROAS, TH-BKK-ROAS (extended ACL)

CCNA 200-301: Domain 5.0 (Security Fundamentals)

Security+ SY0-701: Domain 3.0 (Security Architecture — network segmentation & access control)

Purpose: the largest objective in this phase. Part A restricts management access; Part B contains guest traffic — see Objective 1's policy design for the full rationale.

Part A — Standard ACL, word by word

```
ip access-list standard MGMT-SSH-ONLY
 permit 10.10.10.0 0.0.0.255
 permit 10.10.110.0 0.0.0.255
 permit 10.10.120.0 0.0.0.255
!
line vty 0 15
 access-class MGMT-SSH-ONLY in
!
ipv6 access-list MGMT-SSH-ONLY-V6
 permit ipv6 2001:DB8:1:10::/64 any
 permit ipv6 2001:DB8:3:10::/64 any
 permit ipv6 2001:DB8:4:10::/64 any
!
line vty 0 15
 ipv6 access-class MGMT-SSH-ONLY-V6 in
```

A REAL SYNTAX CORRECTION, NOT A PLATFORM QUIRK

Unlike IPv4, Cisco IOS has no separate 'standard' IPv6 ACL type - every `ipv6 access-list` entry uses extended-style syntax, which always requires a protocol keyword before the source prefix. The `ipv6` keyword here plays the same role `ip` does in an IPv4 extended ACL - "any IP traffic," as opposed to naming a specific protocol like `tcp` or `udp`. Omitting it is rejected by real IOS, not just this lab platform.

Word / term	Meaning
`ip access-list standard`	A standard ACL only ever matches on source IP address — no destination, no protocol, no port. Sufficient here since the entire policy only needs to ask "did this come from a MGMT subnet or not."
Wildcard mask (`0.0.0.255`)	Same inverse-of-subnet-mask logic covered in Phase 2's OSPF objective — `0` bits must match exactly, `1` bits don't care.
`access-class ... in`	Applies an ACL specifically to the VTY (virtual terminal) lines, filtering inbound connection attempts — different from `ip access-group` , which applies an ACL to a regular data-plane interface.

> **TEST CAREFULLY BEFORE DISCONNECTING**
>
> A standard ACL matching only source address is a classic way to accidentally lock yourself out if your own management session doesn't originate from one of the permitted subnets. Verify continued access (or console availability as a fallback) before ending the session used to apply this.

Part B — Extended ACL, word by word

```
ip access-list extended GUEST-CONTAINMENT
 deny ip 10.10.30.0 0.0.0.255 10.10.10.0 0.0.0.255
 deny ip 10.10.30.0 0.0.0.255 10.10.20.0 0.0.0.255
 deny ip 10.10.30.0 0.0.0.255 10.10.40.0 0.0.0.255
 permit ip any any
!
interface Vlan30
 ip access-group GUEST-CONTAINMENT in
```

Word / term	Meaning
`ip access-list extended`	Can match source AND destination address, protocol, and port — required here since the policy depends on both where traffic originates (GUEST_WIFI) and where it's headed (MGMT/LOGISTICS_SALES/DMZ_SERVERS).
Implicit deny + explicit `permit ip any any`	Every ACL ends with an invisible `deny any any` if nothing else matches — the explicit `permit ip any any` at the end of this list overrides that default, allowing every other kind of traffic this ACL doesn't specifically care about (e.g. guest-to-internet, once that path exists in Phase 4).
`ip access-group ... in`	Applies an ACL to a regular routed interface, in the inbound direction relative to that interface — filtering as close to the traffic's source as possible is standard placement practice for extended ACLs.

The IPv6 equivalents use `ipv6 access-list` for the definition and `ipv6 traffic-filter` (not `ip access-group`) to apply it to an interface, and `ipv6 access-class` (not `access-class`) on the VTY lines — IPv6's

ACL commands are named distinctly from their IPv4 counterparts throughout.

EXPECTED RESULT

```
show access-lists
show ip interface Vlan30
```

Confirms the ACL is applied inbound on the correct interface, and — once test traffic exists — that a GUEST_WIFI-sourced packet toward a protected subnet actually gets denied, visible as the matching `deny` line's counter incrementing in `show access-lists`.

11. Objective 9 — PKI: Internal CA, CSR, Signed Certificate

Not a Packet Tracer task — real OpenSSL commands executed directly

Security+ SY0-701: Domain 1.0 (General Security Concepts — PKI)

AWS SAA-C03: Design Secure Architectures (certificate management for HTTPS/ALB listeners)

Purpose: demonstrate the full chain-of-trust workflow a real enterprise uses for internally-issued TLS certificates — standing up a Certificate Authority, generating a Certificate Signing Request for a server, and signing it with that CA — using real OpenSSL output rather than a simulated placeholder.

Commands, word by word

```
openssl genrsa -out ca-root-key.pem 4096
openssl req -x509 -new -nodes -key ca-root-key.pem -sha256 -days 3650 -out ca-root-cert.pem -
subj "..."
openssl genrsa -out ecommerce-server-key.pem 2048
openssl req -new -key ecommerce-server-key.pem -out ecommerce-server.csr -config ecommerce-
server-san.cnf
openssl x509 -req -in ecommerce-server.csr -CA ca-root-cert.pem -CAkey ca-root-key.pem -
CAcreateserial \
  -out ecommerce-server-cert.pem -days 825 -sha256 -extfile ecommerce-server-san.cnf -
extensions v3_req
openssl verify -CAfile ca-root-cert.pem ecommerce-server-cert.pem
```

Word / term	Meaning
CA (Certificate Authority)	An entity trusted to vouch for the identity behind a public key, by digitally signing a certificate binding that key to a name (here, `CA-ROOT.danny-sec.internal`). Real browsers/OSes trust a small set of public root CAs by default; this is a private, internal-only equivalent.
`genrsa`	Generates an RSA private key — 4096-bit for the CA (root keys warrant extra strength since they sign everything else), 2048-bit for the server (the current practical minimum for real security).
`req -x509`	Instead of producing an unsigned request, directly produces a **self-signed** certificate — appropriate only for a root CA, which by definition has no higher authority to sign it instead.
CSR (Certificate Signing Request)	An unsigned "ask" — contains the server's public key and identity information, but carries no trust on its own until a CA signs it.
SAN (Subject Alternative Name)	A certificate extension listing every DNS name/IP address the certificate is actually valid for. Modern browsers ignore the older CN-only identification method entirely and check SAN instead — a certificate without SAN entries is effectively broken for real-world use, which is why the CSR here was built via a config file rather than the simpler `-subj`-only method.
`x509 -req ... -CA ... -CAkey ...`	The actual signing operation — the CA's private key is used to cryptographically sign the CSR's contents, producing the final trusted certificate. `-CAcreateserial` has OpenSSL track a serial number for every certificate this CA issues, since two certificates from the same CA must never share a serial number.
`verify -CAfile`	Confirms the chain of trust actually works — that the signature on the server certificate really was produced by this specific CA's key, not tampered with or signed by something else.

EXPECTED RESULT

```
ecommerce-server-cert.pem: OK
```

Confirms the signature chain validates correctly.

IMPORTANT CAVEAT, STATED EXPLICITLY IN THE REPO

These private keys are committed to a public GitHub repository intentionally, purely to demonstrate the workflow end-to-end — they protect nothing real, since no live server anywhere uses them and this CA has no relationship to any actual trusted root. A private key protecting anything genuine should never leave the machine it was generated on, let alone enter version control.

12. Objective 10 — TCP vs UDP Capture (Packet Tracer Simulation Mode)

Not a config task — manual capture within Packet Tracer's GUI

CCNA 200-301: Domain 1.0 (Network Fundamentals — TCP vs UDP characteristics)

Purpose: directly observe the difference between a connection-oriented and a connectionless transport protocol, using this lab's own real (simulated) traffic rather than a textbook diagram.

Term	Meaning
TCP (Transmission Control Protocol)	Connection-oriented — establishes a session via a **three-way handshake** (`SYN` , `SYN-ACK` , `ACK`) before any application data flows, and tracks sequence numbers to guarantee ordered, reliable delivery. SSH runs over TCP port 22, which is exactly why it's the example used here.
UDP (User Datagram Protocol)	Connectionless — no handshake, no delivery guarantee, no ordering guarantee. A datagram is simply sent; if it's lost, nothing at the transport layer notices or retries. Syslog and SNMP both run over UDP, which is why they're the contrasting example.

WHY REAL WIRESHARK ISN'T THE TOOL HERE

Real Wireshark can only see traffic on your host machine's actual network interface — it has no visibility into Packet Tracer's internally simulated packets, which never touch a real NIC. Packet Tracer's own **Simulation Mode** is the correct, and only, way to inspect this specific lab's actual packet exchanges in real protocol detail.

Capture procedure

1. Switch Packet Tracer to Simulation Mode.
2. Start an SSH session between two devices (e.g. `ssh -l asean.admin 10.10.110.1` from `PH-MNL-ACC`) and step through the captured events, opening each SSH-related packet's PDU detail to see the explicit `SYN` / `SYN-ACK` / `ACK` flag sequence.
3. Trigger a UDP event (an SNMP poll or a Syslog message) and open its PDU detail — notice there is no handshake at all, just a single datagram with no connection state.

EXPECTED RESULT

Two contrasting PDU-detail screenshots — one showing TCP's explicit 3-packet handshake, one showing UDP's single unacknowledged datagram — saved into `02-security-hardening/threat-`

> `simulations/` with captions explaining the contrast.

13. Platform Lessons Learned

Observation	Resolution
SNMPv3 (`group` / `user` / `host` / `enable traps`) entirely absent — only `community` exists	Confirmed via direct `snmp-server ?` query. Reference files kept accurate to real hardware; live lab uses `snmp-server community AREHI-RO-2026 RO` instead, documented as a deliberate, explicit downgrade rather than silently accepted
NTP peer-address discrepancy on `SG-EDGE-GW` (synced via an unconfigured directly-connected address rather than the configured loopback)	Functionally synchronized either way, confirmed via `show ntp status` ; the specific-peer oddity treated the same way as other unexplained-but-harmless simulator quirks
No dedicated `show snmp` command family on this platform	Fell back to `show running-config | include snmp` , the same universal fallback used whenever a dedicated verification command turned out not to exist

14. Appendix — Verification Command Quick Reference

What to check	Command
NTP sync state	`show ntp status`, `show ntp associations`
SNMP (lab fallback)	`show running-config | include snmp`
DHCP Snooping / DAI state	`show ip dhcp snooping`, `show ip arp inspection`
DHCP relay config	`show run interface <vlan>`
NAT translations	`show ip nat translations`, `show ip nat statistics`
ACL application & hit counters	`show access-lists`
PKI chain of trust	`openssl verify -CAfile ca-root-cert.pem ecommerce-server-cert.pem`
